

Chapter 12

What Stories Are Not

To help us better understand what user stories are, it's important to look at what they are not. This chapter explains how user stories differ from three other common approaches to requirements: use cases, IEEE 830 software requirements specifications, and interaction design scenarios.

User Stories Aren't IEEE 830

The Computer Society of the Institute of Electrical and Electronics Engineers (IEEE) has published a set of guidelines on how to write software requirements specifications (IEEE 1998). This document, known as IEEE Standard 830, was last revised in 1998. The IEEE recommendations cover such topics as how to organize the requirements specification document, the role of prototyping, and the characteristics of good requirements. The most distinguishing characteristic of an IEEE 830-style software requirements specification is the use of the phrase “The system shall...” which is the IEEE's recommended way to write functional requirements. A typical fragment of an IEEE 830 specification looks similar to the following:

- 4.6) The system shall allow a company to pay for a job posting with a credit card.
 - 4.6.1) The system shall accept Visa, MasterCard and American Express cards.
 - 4.6.2) The system shall charge the credit card before the job posting is placed on the site.
 - 4.6.3) The system shall give the user a unique confirmation number.

Documenting a system's requirements to this level is tedious, error-prone, and very time-consuming. Additionally, a requirements document written in this way is, quite frankly, boring to read. Just because something is boring to

read is not sufficient reason to abandon it as a technique. However, if you're dealing with 300 pages of requirements like this (and that would only be a medium-sized system), you have to assume that it is not going to be thoroughly read by everyone who needs to read it. Readers will either skim or skip sections out of boredom. Additionally, a document written at this level will frequently make it impossible for a reader to grasp the big picture.

A Warning Sign

One warning sign of a project going astray with a requirements specification is a ping-ponging of the specification document between the software development group and another group like Marketing or Product Management. What typically happens is the Product Management (or similar) group writes a requirements specification that is given to the developers. The developers then rewrite this document so that it conveys their interpretation of the requirements as first written by Product Management. The developers are always careful to give their document a completely different name (something like Functional Specification perhaps) to hide that it is the same document as the initial document, just written from the perspective of a different group.

Both groups know that a requirements specification for a project of any significance is too difficult to read and fully understand and impossible to write with the desired precision. So, whichever group writes the final requirements can claim ownership of the intent of the document. When the project is finished and blame is being allocated they will point to sections of the document and claim that missing features were implied. Or they will claim that expected functionality is clearly out of scope because of a sentence buried somewhere in the document.

Most of the times when I see two groups writing separate versions of essentially the same document I already know they are positioning themselves for the end-of-project blame sessions and for claiming to know the intent of the document. This type of silliness goes away with user stories. Along with the shift to conversations from documentation comes the freedom of knowing that nothing is final. Documents that look like contracts feel so final. Conversations don't feel that way. If we talk today and then learn something next month, we talk again.

There is a tremendous appeal to the idea that we can think, think, think about a planned system and then write all the requirements as "The system shall..." That sounds so much better than "if possible, the system will..." or even "if we have time, we'll try to..." that better characterizes the reality on most projects.

Unfortunately, it is effectively impossible to write all of a system's requirements this way. There is a powerful and important feedback loop that occurs when users see the software being built for them. When users see the software,

they will come up with new ideas and change their minds about old ideas. When changes are requested to the software described in a requirements specification, we've become accustomed to calling it a "change of scope." This type of thinking is incorrect for two reasons. First, it implies that the software was at some point sufficiently well-known for its scope to have been considered fully defined. It doesn't matter how much effort is put into upfront thinking about requirements, we've learned that users will have different (and better) opinions once they see the software. Second, this type of thinking reinforces the belief that software is complete when it fulfills a list of requirements, rather than when it fulfills its intended users' goals. If the scope of the user's goals changes then perhaps we can speak of a "change of scope," but the term is usually applied even when it is only the details of a specific software solution that have changed.

IEEE 830-style requirements have sent many projects astray because they focus attention on a checklist of requirements rather than on the user's goals. Lists of requirements do not give the reader the same overall understanding of a product that stories do. It is very difficult to read a list of requirements without automatically considering solutions in your head as you read. Carroll (2000) suggests that designers "may produce a solution for only the first few requirements they encounter." For example, consider the following requirements:¹

- 3.4) The product shall have a gasoline-powered engine.
- 3.5) The product shall have four wheels.
 - 3.5.1) The product shall have a rubber tire mounted to each wheel.
- 3.6) The product shall have a steering wheel.
- 3.7) The product shall have a steel body.

By this point I suppose images of an automobile are floating around your head. Of course, an automobile satisfies all of the requirements listed above. The one in your head may be a bright red convertible while I might be envisioning a blue pickup. Presumably the differences between your convertible and my pickup are covered in additional requirements statements.

But suppose that instead of writing an IEEE 830-style requirements specification, the user told us her goals for the product:

- The product makes it easy and fast for me to mow my lawn
- I am comfortable while using the product

1. Adapted from *The Inmates Are Running the Asylum* (Cooper 1999).

By looking at the user's goals, we get a completely different view of the product and realize that the customer really wants a riding lawn mower, not an automobile. These goals are not user stories, but where IEEE 830 documents are a list of requirements, stories describe a user's goals. By focusing on the user's goals for the new product, rather than a list of attributes of the new product, we are able to design a better solution to the user's needs.

A final difference between user stories and IEEE 830-style requirements specifications is that with the latter the cost of each requirement is not made visible until all the requirements are written down. The typical scenario is that one or more analysts spends two or three months (often longer) writing a lengthy requirements document. This is then handed to the programmers who tell the analysts (who relay the message to the customer) that the project will take twenty-four months, rather than the six months they had hoped for. In this case, time was wasted writing the three-fourths of the document that the team won't have time to develop, and more time will be wasted as the developers, analysts and customer iterate over which functionality can be developed in time. With stories, an estimate is associated with each story right up front. The customer knows the velocity of the team and the story point cost of each story. After writing enough stories to fill all the iterations, she knows she's done.

Kent Beck explains this difference with an analogy of registering for a wedding.² When you register for a wedding you don't see the cost of each item. You just make a wish list of everything you want. That may work for weddings, but it doesn't work for software development. When a customer places an item on her project wish list, she needs to know the cost of it.

▼

How and Why Will the Feature Be Used?

A further problem with requirements lists is that the items on the lists describe the behavior of the software, not the behavior or goals of a user. A requirements list rarely answers "But how and why will someone use this feature?"

Steve Berczuk, XP developer and author of *Software Configuration Management Patterns*, points out the importance of this question: "I can't count the number of times that I saved a whole lot of work by taking a feature list and asking a customer to create scenarios that use those features. Often the customer will realize that the feature really isn't needed, and that you should spend time building things that add value."³

▲

2. Personal communication, November 7, 2003.

3. Steve Berczuk on extremeprogramming@yahoo.com, February 20, 2003.

User Stories Are Not Use Cases

First introduced by Ivar Jacobsen (1992), use cases are today most commonly associated with the Unified Process. A use case is a generalized description of a set of interactions between the system and one or more actors, where an actor is either a user or another system. Use cases may be written in unstructured text or to conform with a structured template. The templates proposed by Alistair Cockburn (2001) are among the most commonly used. A sample is shown in Figure 12.1, which is equivalent to the user story “A Recruiter can pay for a job posting with a credit card.”

Since this is not a book on use cases we won’t fully cover all the details of the use case shown in Figure 12.1; however, it is worth reviewing the meaning of the Main Success Scenario and Extensions sections. The Main Success Scenario is a description of the primary successful path through the use case. In this case, success is achieved after completing the five steps shown. The Extensions section defines alternative paths through the use case. Often, extensions are used for error handling; but, extensions are also used to describe successful but secondary paths, such as in extension 3a of Figure 12.1. Each path through a use case is referred to as a *scenario*. So, just as the Main Success Scenario represents the sequence of steps one through five, an alternate scenario is represented by the sequence 1, 2, 2a, 2a1, 2, 3, 4, 5.

One of the most obvious differences between stories and use cases is their scope. Both are sized to deliver business value, but stories are kept smaller in scope because we place constraints on their size (such as no more than ten days of development work) so that they may be used in scheduling work. A use case almost always covers a much larger scope than a story. For example, looking at the user story “A Recruiter can pay for a job posting with a credit card” we see it is similar to the main success scenario of Figure 12.1. This leads to the observation that a user story is similar to a single scenario of a use case. Each story is not necessarily equivalent to a main success scenario; for example, we could write the story “When a user tries to use an expired credit card the system prompts her to enter a different credit card,” which is equivalent to Extension 2b of Figure 12.1.

User stories and use cases also differ in the level of completeness. James Grenning has noted that the text on a story card “plus acceptance tests are basically the same thing as a use case.”⁴ By this Grenning means that the story cor-

4. James Grenning on extremeprogramming@yahoo.com, February 23, 2003.

Use Case Title: Pay for a job posting
Primary Actor: Recruiter
Level: Actor goal
Precondition: The job information has been entered but is not viewable.
Minimal Guarantees: None
Success Guarantees: Job is posted; recruiter's credit card is charged.
Main Success Scenario:

1. Recruiter submits credit card number, date, and authentication information.
2. System validates credit card.
3. System charges credit card full amount.
4. Job posting is made viewable to Job Seekers.
5. Recruiter is given a unique confirmation number.

Extensions:

2a: The card is not of a type accepted by the system:
2a1: The system notifies the user to use a different card.

2b: The card is expired:
2b1: The system notifies the user to use a different card.

2c: The card is expired:
2c1: The system notifies the user to use a different card.

3a: The card has insufficient available credit to post the ad.
3a1: The system charges as much as it can to the current credit card.
3a2: The user is told about the problem and asked to enter a second credit card for the remaining charge. The use case continues at Step 2.

Figure 12.1 *A sample use case for pay for a job posting.*

responds to the use case's main success scenario, and that the story's tests correspond to the extensions of the use case.

For example, in Chapter 6, "Acceptance Testing User Stories," we saw that appropriate acceptance test cases for the story "A Recruiter can pay for a job posting with a credit card" might be:

- Test with Visa, MasterCard and American Express (pass).
- Test with Diner's Club (fail).
- Test with good, bad and missing card ID numbers.

- Test with expired cards.
- Test with different purchase amounts (including one over the card's limit).

Looking at these acceptance tests we can see the correlation between them and the extensions of Figure 12.1.

Another important difference between use cases and stories is their longevity. Use cases are often permanent artifacts that continue to exist as long as the product is under active development or maintenance. Stories, on the other hand, are not intended to outlive the iteration in which they are added to the software. While it is possible to archive story cards, many teams simply rip them up.

An additional difference is that use cases are more prone to including details of the user interface, despite admonishments to avoid this (Cockburn 2001; Adolph, Bramble, et al. 2002). There are several reasons this happens. First, use cases often lead to a large volume of paper and without another suitable place to put user interface requirements they end up in the use cases. Second, use case writers focus too early on the software implementation rather than on business goals.

Including user interface details causes definite problems, especially early on in a new project when user interface design should not be made more difficult by preconceptions. I recently came across the use case shown in Figure 12.2, which describes the steps for composing and sending an email message.

Use Case Title: Compose and send email message

Main Success Scenario:

1. User selects the "New Message" menu item.
2. System presents the user with the "Compose New Message" dialog.
3. User edits email body, subject field and recipient lines.
4. User clicks the Send button.
5. System sends the message.

Figure 12.2 *A use case to compose and send an email message.*

This use case has user interface assumptions throughout. It assumes that there is a "New Message" menu item, that there is a dialog for composing new messages, that there are subject and recipient input fields on that dialog, and that there is a Send button. Many of these assumptions may seem good and safe but they may rule out a user interface where I click on a recipient's name to ini-

tiate the message instead of typing it in. Additionally, the use case of Figure 12.2 precludes the use of voice recognition as the interface to the system.

Admittedly, there are far more email clients that work with typed messages than with voice recognition, but the point is that a use case is not the proper place to specify the user interface like this. Think about the user story that would replace Figure 12.2: “A user can compose and send email messages.” No hidden user interface assumptions there. With stories, the user interface will come up during the conversation with the customer.

To get around the problem of user interface assumptions in use cases, Constantine and Lockwood (1999) have suggested the use of *essential use cases*. An essential use case is a use case that has been stripped of hidden assumptions about technology and implementation details. For example, Table 12.1 shows an essential use case for composing and sending an email message. What is interesting about essential use cases is that the user intentions could be directly interpreted as user stories.

Another difference is that use cases and stories are written for different purposes (Davies 2001). Use cases are written in a format acceptable to both customers and developers so that each may read and agree to them. Their purpose is to document an agreement between the customer and the development team. Stories, on the other hand, are written to facilitate release and iteration planning, and to serve as placeholders for conversations about the users’ detailed needs.

Table 12.1 *An essential use case.*

User Intention	System Responsibility
Compose email message	
Indicate recipient(s)	
	Collect email content and recipient(s)
Send email message	
	Send the message

Not all use cases are written by filling in a form, as shown in Figure 12.1. Some use cases are written as unstructured text. Cockburn refers to these as *use case briefs*. Use case briefs differ from user stories in two ways. First, since a use case brief must still cover the same scope as a use case, the scope of a use case brief is usually larger than the scope of a user story. That is, one use case brief will typically tell more than one story. Second, use case briefs are intended to live on for the life of a product. User stories, on the other hand, are disposed of.

Finally, use cases are generally written as the result of an analysis activity, while user stories are written as notes that can be used to initiate analysis conversations.

User Stories Aren't Scenarios

In addition to referring to a single path through a use case, the word *scenario* is also used by human-computer interaction designers. In this context a scenario is a detailed description of a user's interaction with a computer. The scenarios of interaction design are not the same as a scenario of a use case. In fact, an interaction design scenario is typically larger, or more encompassing, than even a use case. For example, consider this scenario:

Maria is thinking about making a career change. Since the glory days of the dot-com boom she has worked as a tester at BigTechCo. A former high school math teacher, Maria decides she'll be happier if she returns to teaching. Maria goes to the BigMoneyJobs.com website. She creates a new account with a user name and password. She then creates her resume. She wants to find a job as a math teacher anywhere in Idaho but preferably near her current job in Coeur d'Alene. Maria finds a handful of jobs that match her search criteria. The job that intrigues her most is with the North Shore School, a private high school in Boise. Maria has a friend, Jessica, in Boise whom she hopes may know someone at North Shore. Maria enters Jessica's email address and forwards the job link to her with a note asking if she knows anyone at the school. The next morning Maria gets an email from Jessica saying that she doesn't know anyone at the school, but she knows of the North Shore School and it has a wonderful reputation. Maria clicks on a button that submits her resume to North Shore.

Carroll (2000) says that scenarios include the following characteristic elements:

- a setting
- actors
- goals or objectives
- actions and events

The setting is the location where the story takes place. In the story about Maria the story presumably takes place on her home computer; but since that is not stated, the location of the story could be her office during the workday.

Each scenario includes at least one actor. It is possible for a scenario to have multiple actors. For example, in our scenario both Maria and Jessica are actors. Maria may be referred to as the *primary actor* because the scenario mostly describes her interactions with the system. However, because Jessica receives an email from the system and then uses the website to look at the job posting, she is considered a *secondary actor*. Unlike use cases, actors in interaction design scenarios are always people and never other systems.

Each actor in a scenario is pursuing one or more goals. As with actors, there can be primary and secondary goals. For example, Maria's primary goal is to find an appropriate job in her desired location. While working to achieve that goal, she pursues secondary goals such as viewing detailed information about a hotel or sharing information with a friend.

Carroll refers to the actions and events as the *plot* of a scenario. They are the steps an actor takes to achieve her goal or a system's response. Searching for a job in Idaho is an action Maria performs. The response to that action is the event of the system displaying a list of matching jobs.

The primary differences between user stories and scenarios are scope and detail. Scenarios contain much more detail and their scope usually covers multiple stories. The example scenario contains many possible stories, such as:

- a user can send information about a job to a friend via email
- a user can create a resume
- a user can submit her resume for a matching job
- a user can search for a job by geographic region

Even with their additional detail, scenarios (like stories) leave details to be worked through in a discussion. For example:

- Maria logged onto the site with a user name and password. Are all users required to log onto the site? Or does logging on enable some of the features Maria used (perhaps the feature to send an email)?
- When Jessica receives the email, does the email contain information about the job or does it just link to a page on the site with that information?

Summary

- User stories are different from IEEE 830 software requirements specifications, use cases and interaction design scenarios.
- No matter how much thinking, thinking and thinking we do, we cannot fully and perfectly specify a non-trivial system upfront.
- There is a valuable feedback loop that occurs between defining requirements and users having early and frequent access to the software.
- It is more important to think about users' goals than to list the attributes of a solution.
- User stories are similar to a use case scenario. But use cases still tend to be larger than a single story and can be more prone to containing embedded assumptions about the user interface.
- Additionally, user stories differ from use cases in their completeness and longevity. Use cases are much more complete than are user stories. Use cases are designed to be permanent artifacts of the development process; user stories are more transient and not intended to outlive the iteration in which they are developed.
- User stories and use cases are written for different purposes. Use cases are written so that developers and customers can discuss them and agree to them. User stories are written to facilitate release planning and to serve as reminders to fill in requirements details with conversations.
- Unlike IEEE 830 specifications and use cases, user stories are not artifacts of analysis activities. Rather, user stories are a support tool for analysis.
- Interaction design scenarios are much more detailed than user stories and differ in the kind of detail provided.
- A typical interaction design scenario is much larger than a user story. One scenario may comprise multiple use cases, which, in turn, may comprise many user stories.

Questions

- 12.1 What are the key differences between user stories and use cases?

- 12.2 What are the key differences between user stories and IEEE 830 requirements statements?
- 12.3 What are the key differences between user stories and interaction design scenarios?
- 12.4 For a non-trivial project, why is it impossible to write all the requirements at the start of the project?
- 12.5 What is the advantage to thinking about users' goals rather than on listing the attributes of the software to be built?